The Gateway Is Not the Platform: The Operational Cost of Taking AI On-Premises

Kostadis Roussos
Independent researcher
kostadis@gmail.com

October 25, 2026

Abstract

Taking an AI workload on-premises is usually framed as a model decision: which open-weight model fits, how fast it decodes, how close it comes to a frontier API. We argue it is an operations decision. Swapping a hosted API for a local model means tuning three layers, not one: the model and its serving, your own code, and every tool you depend on. Each layer is a recurring operational cost, and an API-compatible gateway hides all three, because everything connects on the first try.

We report three tools moved onto a two-node NVIDIA DGX Spark cluster serving open-weight models through vLLM and Ollama:

- **Cognee** (knowledge-graph memory, 1.2.2 and later `main`) on a 106-file, 55,000-word tabletop-campaign corpus;
- **MemPalace** (semantic memory store, a fork of v3.10.0) on several campaign palaces;
- **zvec-grep** (hybrid workspace search, a fork of v0.2.1) indexing code and documents.

All three reached local models through an OpenAI-compatible endpoint within minutes. None worked at scale without further work:

- Cognee failed on about six attempts in one day, each in a different way: unbounded fan-out against fixed serving slots, timeouts that counted queue wait, retry layers that turned a 600 s timeout into 1,801 s, all-or-nothing durability that discarded 50 minutes of work, and no liveness signal. A later version completed in 27 minutes; a reasoning model then broke it again by spending its whole output budget thinking.
- MemPalace needed about a dozen changes on a fork over five months, while upstream releases were merged in on top.
- zvec-grep needed eleven commits on a fork (about 1,800 lines) to add a local embedding provider with a 30-minute timeout, one request in flight, and per-request batch limits.

The fixes cluster into the same few categories across all three tools: capacity, liveness, per-model behavior, request limits, where compute and state live, scope, and configuration authority.

One operator made those changes, working with AI coding agents: 70 of the 76 MemPalace fork commits and 59 of the 69 commits in the repository that holds our local control plane carry a coding-agent co-author, and all 11 zvec-grep fork commits were written with an agent. A cost that once argued for waiting on upstream, or for staying on the hosted API, became work one person could carry; the zvec-grep adaptation took two days.

We argue four things:

1. Cloud-built tools inherit a control plane they never see. Going on-prem removes it.

2. An API-compatible gateway restores the wire protocol but not that control plane, so it is necessary but insufficient. It also hides the gap, which is why the cost arrives late.
3. The real cost of on-premises AI is operational: tuning the model, your code, and your tools, and paying that again on every model swap and every upstream release.
4. AI coding agents change the economics of that cost. They make adapting someone else's tool, and carrying the fork, cheap enough for one person to do. They do not remove the cost; they move it from writing code to deciding what to change and checking that it worked.

1 Introduction

The usual story of on-premises AI is about models: which open-weight model fits in memory, how fast it decodes, and how close its quality comes to a frontier API. That story assumes the rest of the stack carries over. Once a local server speaks the OpenAI chat-completions and embeddings protocols, the tools built on those protocols should work.

They connect. They do not necessarily work. A hosted model API is more than an endpoint. Behind it sits a control plane the client never sees:

- it admits requests elastically, so fan-out width is someone else's problem;
- it answers in seconds, so a timeout measures failure, not queue depth;
- it is always up, so the only health check needed is "did the call return";
- it accepts any reasonable request size;
- its models behave as documented, so an output budget means output;
- it runs somewhere else, so where compute and state live is not the tool's concern.

Tools built against that API never needed to model any of this, so they don't.

This paper reports what happened when we moved three such tools, Cognee [1], MemPalace [4] and zvec-grep [6], onto two DGX Spark boxes. We make four claims:

1. **Going on-prem means tuning three layers.** The model and its serving, your own code, and the tools you depend on. The model was the layer that worked best.
2. **A gateway is necessary but insufficient.** An OpenAI-compatible endpoint, and a router such as LiteLLM in front of it, restores the wire protocol. It does not restore admission, liveness, request limits or per-model behavior. Worse, it makes a local server look like the cloud, so the gap shows up only under load, after the decision to go on-prem has been made.
3. **The cost is operational and recurring.** Each model swap re-opens layer 1. Each upstream release of a forked tool re-opens layer 3. The local control plane we built (layer 2) is what keeps that cost bounded.
4. **Coding agents change who can pay it.** Every fix in this paper was written by one person working with AI coding agents. Adapting a tool to local infrastructure, and keeping the fork current, was something one person could do alongside everything else. That changes the build-versus-wait-versus-stay-hosted decision, not the existence of the cost.

The evidence is one operator, three tools and one cluster. Section 10 discusses what that does and does not support.

Table 1: Tool versions used in this study.

Tool	Version	Notes
Cognee	1.2.2 (PyPI, released 2026-06-26)	Runs of 2026-07-03 to 07-04
Cognee	`main` at `e61bbb79c` (2026-08-01)	Successful retest, editable install
Cognee	`main` at `e8dd3d93d` (pulled 2026-09-02)	Reasoning-model retest of 2026-09-10
MemPalace	fork `kostadis/mempalace`, branch `kostadis-dev`, at `d23d6a7` (2026-10-24)	Based on upstream v3.10.0
zvec-grep	fork `kostadis/zvec-grep` of `zvec-ai/zvec-grep`, at `73f6991` (2026-09-09)	Based on upstream v0.2.1

2 Background

2.1 Hardware and serving

The cluster is two NVIDIA DGX Spark boxes (GB10, 128 GB of unified CPU/GPU memory, about 273 GB/s of memory bandwidth). Each serves one chat model through vLLM's OpenAI-compatible API. Its batch width is fixed by `max_num_seqs`, which we ran at 3, 8 or 16 depending on the configuration.

Embeddings come from Ollama serving `qwen3-embedding:0.6b` (1024 dimensions). Decode is bandwidth-bound, so a long structured output is the most expensive thing a client can ask for.

The chat models varied across the study. The first runs (2026-07-04) used Qwen3-Next-80B-A3B-Instruct-FP8. The August retest used DeepSeek-V4-Flash-DSpark across both boxes. The September retest used the reasoning model `qwen3.8-flash-next`.

2.2 The three tools

Cognee builds a knowledge graph from documents and answers questions over it. `remember()` runs `add()` (store raw text), then `cognify()` (all the LLM work), then an optional self-improvement loop, which we disabled. For each document, `cognify()` classifies it, chunks it, and runs `extract_graph_and_summarize`: two structured-output chat calls per chunk. It then embeds the results and writes them to three embedded stores: Kuzu for the graph (wrapped as "Ladybug" and run as a subprocess), LanceDB for vectors and SQLite for metadata. Cognee reaches the model through LiteLLM and enforces output structure through Instructor.

MemPalace is a semantic memory store. It mines files into "drawers", embeds them, and serves semantic search over one or more "palaces". Upstream embeds on the machine that runs it, with a bundled ONNX model, and stores vectors in ChromaDB.

zvec-grep (`zg`) is a hybrid code-and-document search tool for coding agents, exposed as a CLI and an MCP server. Upstream embeds in-process with a local model or calls a hosted embedding provider.

2.3 Versions

Table 1 lists the exact versions used.

2.4 Workloads

The Cognee corpus is the grounding documents of a running tabletop campaign: three or four dense state documents of 2,000 to 4,800 words each, plus 103 or 104 NPC dossiers of about 230 words each. That is 106 to 108 files and about 50,000 to 55,000 words. A single dense document produces graph-extraction outputs of thousands of tokens per chunk.

MemPalace mines campaign checkouts on a Linux VM into several per-campaign palaces plus a chat palace, embedding on Ollama on the machine that hosts the VM. zvec-grep indexes local workspaces of code and documents against the Spark's Ollama embedding endpoint.

Cognee is chat-heavy: its cost is long structured decodes. MemPalace and zvec-grep are embedding-heavy: their cost is many large embedding requests. That difference matters, because the same gaps appeared in both.

3 Why it looks cheap

Connecting each tool to the local stack was easy, and that ease is what misleads.

Cognee needed five settings in `.env`:

- `LLM_PROVIDER="custom"`, which routes through LiteLLM;
- an `openai/`-prefixed model id, which LiteLLM needs to route to `LLM_ENDPOINT` and strips before sending;
- the vLLM endpoint;
- `EMBEDDING_PROVIDER="openai_compatible"`, pointed at Ollama's `/v1`;
- `EMBEDDING_DIMENSIONS="1024"`.

A one-document smoke test then passed: `party.md`, about 1,400 words, 4 LLM calls, 0 retries, 202 s, 61 nodes and 140 edges, and a correct recall answer.

MemPalace, once it had remote embedding providers, switched embedding model with one line of configuration.

zvec-grep documents its local setup as one `curl` check and three commands: register an unauthenticated provider, set the model and endpoint, and index.

By *gateway* we mean this compatibility layer: vLLM's and Ollama's OpenAI-compatible endpoints, plus LiteLLM routing for Cognee. The same reasoning applies to a proxy or router placed in front of them. Without the gateway, nothing connects. With it, everything connects, and nothing yet says what it will cost to run.

Figure 1 shows the split. The tool reaches the model through the gateway. Everything in the right-hand box, the cloud supplied implicitly, and on-premises nothing does unless someone builds it. Sections 4 to 6 are about who built it.

4 Layer 1: tuning the model

The first layer is the one the usual story covers, and it went well.

4.1 Serving calibration

On a single Cognee document the workload is decode-bound: the two graph-extraction calls took 97.5 s and 94.5 s of the 202 s. At batch scale we measured aggregate decode on the real extraction load under three vLLM configurations (Qwen3-Next-80B, spark2), shown in Table 2.

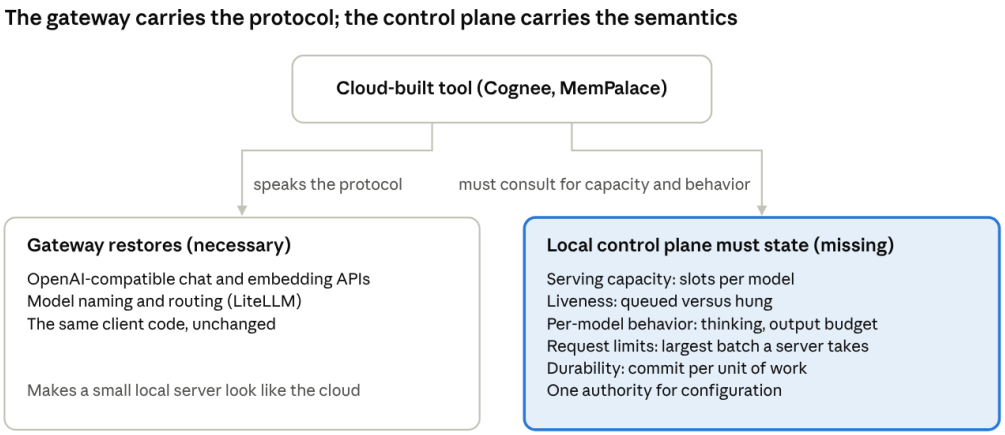


Figure 1: Gateway versus local control plane.

Table 2: Aggregate and per-stream decode on the real graph-extraction load (Qwen3-Next-80B, spark2).

Configuration	Slots	Aggregate (tok/s)	Per stream (tok/s)
MTP-2 speculative decode	3	92	30.7
Plain decode	16	125	7.8
MTP-2 speculative decode	8	138	17.3

Speculative decoding accepted 86 to 93% of draft tokens on the rigid graph JSON, against about 77% on code. That made each MTP stream about 2.3 times faster than a plain one. Eight fast streams beat sixteen bandwidth-contended ones. Prefix caching was immaterial: we measured a hit rate near 0%, and the cacheable prefix was under 3% of a decode-bound call.

This was the study's cleanest result, and it is a result about the server.

4.2 Model behavior is a per-model fact

Three chat models served the study. Each swap changed facts that clients depend on. The clearest case was the reasoning model of September (§6.1): with thinking on, it spent a 16,384-token output budget thinking and wrote nothing. Whether a model thinks, whether that is on by default, and how long it takes to answer are properties of the model, not of the protocol.

4.3 Each swap is an operations event

On this cluster a model swap is not a config change. It means re-running a spin-up script, re-measuring memory headroom, and updating two records in the same change: `current-setup.md`, the inventory of what is served, and `dgxlib/models.yaml`, the registry of how to call it (§5). A hosted API absorbs all of this. On-prem, it recurs with every swap.

5 Layer 2: tuning your own code

What made our tools run was not a better model or a better gateway. It was a set of small, explicit authorities that state what the cloud keeps implicit. We wrote them only because we went on-prem.

- **Per-model behavior: a model registry.** `dgxlib` (in the `dgx-fun` repo [5], 11 commits since 2026-06-19) keeps a registry keyed by served model id: whether the model can think, whether thinking is on by default, its read timeout, and its concurrency. A call site states its intent ("no thinking for this extraction"), and the registry decides whether the model can honor it. Before it existed, each consumer hand-coded these per model. Now swapping a model is a one-line registry edit.
- **Capacity.** The registry exposes per-model serving concurrency, so a client can size its fan-out to the slots that exist instead of discovering them through timeouts.
- **Liveness.** `dgxlib`'s client streams with an idle-token timeout, so it can tell a queued request from a hung one.
- **Configuration authority.** On the mining host, one hand-edited file (`hypostasis.yaml`) declares the vector backend, embedding provider, model and endpoint. Every component's config is rendered from it, so no tool picks its own embedder or endpoint.

None of this is model work, and none of it would exist against a hosted API.

6 Layer 3: tuning your tools

The third layer is the largest and the least visible. Each tool encoded cloud assumptions in different places, and each fix exposed the next. We group the fixes by the assumption each one broke.

6.1 Cognee: a chronology

Before grouping, the Cognee chronology shows how the cost arrives: one failure at a time.

Six attempts, six failures (Cognee 1.2.2, 2026-07-03 to 07-04). Table 3 lists the attempts in order.

The retest that succeeded (2026-08-01). On Cognee `main`, the same corpus completed: `status='completed', items=106, elapsed=1636.3s`, about 27 minutes, with zero errors and zero retries. All five evaluation questions came back correct and grounded in the source. The backend had also changed, to DeepSeek-V4-Flash-DSpark, so this is not a single-variable comparison. Upstream had meanwhile added a backend-aware overload policy and hardened its database workers.

The retest that failed again (2026-09-10). Against a reasoning model the same recipe broke in a new way. Cognee allowed 16,384 output tokens per call, and the model spent all of them thinking. On a 4,000-character input, thinking on took 251 s and produced 0 characters; thinking off took 172 s and produced 25,397. The empty answers produced 493 timeouts, Cognee's automatic rate limiter slowed the run further, and the retries added load. About five GPU-hours went into tuning those symptoms before the cause was found. Disabling thinking through `LLM_ARGS` fixed it: zero timeouts, 104 items, 26.8 minutes. The August recipe had needed three new settings within six weeks.

Table 3: Six attempts to ingest the full corpus with Cognee 1.2.2, 2026-07-03 to 07-04.

Attempt	Outcome	Cause
Latency box (MTP-2, 3 slots), overnight	Died after 3 h 12 min; 253 timeouts	About 39 concurrent requests on a 3-slot queue; retry storm
Throughput box (plain, 16 slots)	Wedged mid-run	One hung request, no liveness signal
MTP-2, 8 slots	Clean for about 50 min, then crashed	Queue-tail requests timed out while waiting, and their retries deepened the queue
Batched `remember()` of 6 documents	First batch ran 61 min, then failed	The four dense documents landed together; retry churn
One document at a time, OS `timeout 600`	Small documents done; dense ones killed	A dense document needs over 10 min of extraction alone
Same, with a 1,800 s timeout	Extraction finished in about 10 min, then wedged	The hang moved to the graph-persist phase

6.2 Capacity and admission

- **Cognee: unbounded fan-out.** Documents are gated by a semaphore of 20, but the per-chunk calls inside a document run under a bare `asyncio.gather` with no cap. That gave about 40 to 48 concurrent requests, so vLLM's `max_num_seqs` was the only throttle: 3 to 15 times overcommitted [2].
- **Cognee: admission paced the wrong thing.** We tested a requests-per-minute limiter, and later Cognee's automatic one. Both made runs worse. They pace how often requests start, while the constraint is how many are in flight, and they reacted to timeouts that were symptoms of something else.
- **Cognee: the gateway disguises locality.** Cognee decides whether a provider is local by its name. Ollama and llama.cpp get a local rate-limit default of 10 requests per minute. vLLM is deliberately excluded, and the code comment says it "absorbs concurrency like a cloud endpoint". That is true of a large vLLM deployment and false of one box with 8 slots. The more faithfully the gateway imitates the cloud, the more surely downstream code treats the server as cloud.
- **zvec-grep: one request in flight.** The fork defaults local indexing to one in-flight embedding request, "to avoid building a server queue and retrying work that may still be running", with an explicit `-embedding-concurrency` knob and back-off to one after transient failures.

Nothing in a chat-completions or embeddings response says "you are request 41 for 8 slots". Capacity is not in the protocol, so every tool had to be told.

6.3 Timeouts, retries and liveness

- **Cognee: timeouts that count queue wait.** The client timer starts at send. A request alive but waiting in a deep queue times out, is retried, and the retry joins the same queue. On a cloud API queue wait is near zero, so this coupling is invisible.
- **Cognee: multiplied retry layers.** LiteLLM retries internally and Instructor retries on top. A 600 s timeout surfaced after 1,801 s, and 1,800 s after 5,401 s.

- **Cognee: no liveness signal.** The only health check is a one-shot 30 s preflight at startup. Non-streaming structured output gives no way to tell "slow" from "hung". Stalls appeared on the chat path, the embedding path and the graph-persist path; in the final wedge the main process sat in `epoll_wait` while the Kuzu workers sat in `pipe_read`.
- **zvec-grep: a timeout sized for the cloud.** Upstream gives hosted providers a 60-second request timeout. The fork allows local embedding requests 30 minutes, "including server queue time and reading the response".

We first attributed the Cognee persist hang to a known orphaned-lock issue [3]. A later investigation showed that attribution was wrong: the issue concerned a different race, and its crash reproducer did not reproduce here. Kuzu operations are serialized by design, so persist time across concurrently admitted documents adds up rather than overlaps, which can look like a hang without being one. Without liveness signals, diagnosis falls back on inference.

6.4 Request limits

- **MemPalace: a request that crashed the server.** The miner sent up to 1,000 inputs in one request to Ollama's `/api/embed`, and Ollama's model runner crashed on large files (#57). The fix was client-side sub-batching to 64.
- **zvec-grep: explicit batch budgets.** The fork caps each local embedding request at 32 fragments and 64,000 characters, across files and within a single large file.

Whether a server survives a large embedding request is a per-server fact. Two tools, written independently, needed the same fix.

6.5 Per-model behavior and identity

- **Cognee: thinking.** The September failure (§6.1) is a per-model fact passed through a gateway unchanged.
- **MemPalace: embedder identity.** Queries and stored drawers must be embedded by the same model. Each provider now reports its own name, so changing model or provider raises an error instead of silently mixing vectors in one palace.
- **zvec-grep: served model ids.** The fork registers a catalog entry for the Spark's `qwen3-embedding:0.6b` (1,024 dimensions, 32,768-token inputs) and resolves served model ids through that catalog, so an index records which model built it.

6.6 Where compute and state live

- **MemPalace: embedding on the wrong machine.** Upstream embeds on the mining machine's CPU with a bundled ONNX model. We wanted the Spark's GPU without installing CUDA wheels on the mining host, which needed remote embedding providers (#6).
- **MemPalace: a hard storage dependency.** ChromaDB was a hard import; a turbovec-only install still pulled it in and broke without it (#20). We added a turbovec backend, later a thin adapter over turbovecdb.
- **Cognee: dependencies behind no gateway.** The graph store is an embedded Kuzu subprocess. No proxy in front of the model touches it.

- **zvec-grep: an extracted backend.** The fork extracted a generic OpenAI-compatible embedding backend so a non-hosted endpoint could be one.

6.7 Scope

MemPalace: one palace per process. Several code paths quietly fell back to a single default palace. Campaign palaces leaked into each other and into the chat palace. The fix touched cross-palace reads, the default palace, a `palace=` argument on every read tool, per-palace tunnels and entity registries, and routing of hook writes (#41, #43, #44, #45, #48, #51). A hosted API has no opinion on this; neither does a gateway.

6.8 Configuration authority and trust

- **Cognee: three silent configuration traps.** `import cognee` calls `load_dotenv(override=True)`, so environment variables set in Python before the import are overwritten; one run meant for spark2 ran on spark1 and its measurements were mislabeled. `.env` is resolved relative to the working directory; from another directory Cognee falls back to OpenAI defaults and fails with an authentication error that names the wrong cause. The `ollama` embedding provider requires `transformers` and a Hugging Face tokenizer download, while `openai_compatible` falls back to TikToken.
- **zvec-grep: authentication assumed.** Upstream assumed every remote provider has an API key. The fork added unauthenticated providers for a trusted local network, and kept authorization to send workspace content as a separate, explicit grant.

Each of these assumes configuration has one obvious home, and on a multi-box local network it doesn't.

6.9 Telling normal from anomalous

- **Cognee: silent fallbacks.** A file path outside `COGNEE_ALLOWED_LOCAL_FILE_ROOTS` is ingested as literal text, and `remember()` still reports `completed`.
- **Cognee: all-or-nothing durability.** One straggler's `InstructorRetryException` propagated through the gather and aborted `remember()`. Fifty minutes of extraction produced an empty graph.
- **MemPalace: log noise.** Every mined file logs a warning that precomputed vectors bypass the configured embedder, though they come from the same model (#59). The vector store records an embedder identity, but it records the function's name (`embed`) rather than the model, and the adapter never reads it, so every open of a populated palace warns (#33).

A tool that cannot tell its normal path from an anomaly is another symptom of the missing control plane.

7 The operational cost

Table 4 collects what each layer cost, using only quantities the working notes recorded. Where nothing was recorded, we say so rather than estimate.

Two features of this cost matter more than its size.

Table 4: Recorded cost of each layer. "Not measured" marks costs the notes did not record.

Layer	Item	Recorded cost
1 Model	Serving calibration	3 vLLM configurations measured on the real load
1 Model	Model swaps	3 chat models over the study; each swap updates `current-setup.md` and `models.yaml`
2 Code	`dgxlib` registry and client	11 commits since 2026-06-19
2 Code	`hypostasis.yaml` configuration authority	Not measured
3 Tools	Cognee 1.2.2	About 6 failed attempts in one day; one run lost 3 h 12 min, another 50 min of extraction
3 Tools	Cognee reasoning-model retest	About 5 GPU-hours on symptoms; 3 new settings six weeks after a working recipe
3 Tools	MemPalace	About a dozen changes on a fork, 2026-05-10 to 10-25, merged under upstream v3.3.6 to v3.10.0
3 Tools	zvec-grep	11 commits on a fork over 2 days, about 1,800 lines added
All	Engineering hours	Not measured

It recurs. A model swap re-opens layer 1 and can re-open layer 3, as the September Cognee retest showed. A forked tool re-opens layer 3 on every upstream release: MemPalace's fork has absorbed upstream releases v3.3.6 through v3.10.0. Carrying a fork is a recurring cost, not a fix.

It is paid late. Every tool passed its smoke test. The gaps appeared under real load: dense documents, large files, many palaces, a different model. By then the decision to go on-prem had been made on the evidence of the smoke test.

Layer 2 is the part that bounds the rest. The registry turned a model swap from an edit in every consumer into one edit. One configuration authority turned "which endpoint is real" from a debugging session into a lookup. But layer 2 only helps tools that consult it. From our failures, a tool meant to run anywhere other than a hosted API should:

1. **Bound fan-out from declared capacity**, not from a semaphore on documents.
2. **Time from progress, not from send**, so it can distinguish queued from hung.
3. **Keep one retry layer** and own it.
4. **Respect declared request limits**, such as the largest batch a server takes.
5. **Commit per unit of work**, so one straggler costs one document, not the run.
6. **Treat model behavior as data** (thinking, output budget, identity), looked up rather than assumed.
7. **Scope every read and write** to an explicit tenant.
8. **Fail loudly on silent fallbacks.**

8 Coding agents change the economics

The cost in Section 7 is real, but it is not the cost it would have been a few years ago. Every change in this paper was made by one operator working with AI coding agents.

Table 5: Operator commits written with a coding agent, by repository.

Repository	Operator commits	With a coding agent	Span
MemPalace fork (`kostadis-dev`)	76	70 (Claude Code)	2026-04-15 to 10-25
`dgx-fun` (holds `dgxlib`, the notes, the spin-up scripts)	69	59 (Claude Code)	through 2026-10-25
zvec-grep fork	11	11 (Codex)	2026-09-08 to 09-09

8.1 The evidence

Table 5 counts the operator's commits in each repository and how many were written with a coding agent.

The MemPalace and `dgx-fun` counts come from co-author trailers naming Claude models used through Claude Code. The zvec-grep commits were written with OpenAI's Codex and carry no trailer; we count them from the operator's account. Two different agents, on a Python and a TypeScript codebase, produced the same kind of result. On Cognee we changed no library code at all; the agent work there was diagnosis, configuration, and the upstream write-up of the failure modes [2].

The zvec-grep fork is the clearest single case: a provider abstraction, a model catalog entry, authentication changes, timeouts, concurrency and batch budgets, with tests and documentation, in eleven commits over two days, against a TypeScript codebase written by another team.

8.2 What changes

Forking becomes a reasonable choice. Without agents, the realistic options for a tool that assumes the cloud are to wait for upstream, to work around it from outside, or to stay on the hosted API. A fork means owning someone else's code, and a fork has to be carried across upstream releases. With agents, one person carried MemPalace across upstream releases v3.3.6 to v3.10.0 while adding about a dozen changes of their own.

Layer 2 becomes cheap to build. `dgxlib`, the configuration authority, and the spin-up scripts are small, specific pieces of infrastructure that a hosted API would have made unnecessary. They are also exactly the kind of code an agent writes well: bounded, testable, and specified by a failure the operator has already seen.

The cost moves rather than disappears. What remained expensive was deciding what to change and checking that it worked. The five GPU-hours of the September Cognee retest went into tuning symptoms before the cause was found. The agents shortened the path from a diagnosis to a working fix, and helped with diagnosis too, but they could reason only from the signals the tools exposed. Without liveness, capacity and identity signals, diagnosis falls back on inference, as the corrected attribution in §6.3 shows. The verification stayed with the operator.

8.3 What it means for the decision

The operational cost in Section 7 recurs. What agents change is its price per occurrence and who can pay it. In our case one person did it: two days for zvec-grep, and a fork carried alongside other

work for five months for MemPalace. That makes on-premises AI viable for operators who could not previously have absorbed the third layer. It does not make it free, and the operator still needs to know what the control plane must hold.

There is an irony worth stating. The agents that made this adaptation cheap, Claude Code and Codex, were themselves hosted models. The local stack was adapted with the help of the cloud.

9 Related work

The failure modes in Section 6 are old distributed-systems problems in new clothes. Timeouts that include queueing, layered retries that amplify load, and the need for per-unit durability are standard lessons of large-service operation. Retry budgets, deadline propagation and backpressure are their usual remedies. What is new is where they surface. LLM tooling grew up behind provider APIs that absorbed these problems, so its tools never adopted the remedies.

We have not run a systematic survey of other memory, RAG, search or agent frameworks, and we make no claim about how common these patterns are. That survey is future work.

10 Limitations

- **One operator, three tools, one cluster.** The evidence is a case study, not a benchmark. The tools differ in workload (chat-heavy and embedding-heavy) and origin, which is some protection against a single tool's quirks, but they were all adapted by the same person.
- **No cloud baseline.** We measured the on-prem cost in absolute terms. We did not run the same workloads against hosted APIs, so we cannot state the cost as a difference.
- **Confounded comparisons.** The Cognee run that succeeded used a newer Cognee and a different chat model than the runs that failed. We cannot attribute the success to either change alone.
- **Unmeasured effort.** Engineering hours were not recorded. Commit and change counts are a proxy, and a weak one.
- **No counterfactual for the agents.** We did not adapt any tool without a coding agent, so we cannot say how much cheaper agents made it. Co-author trailers show that an agent took part in a commit, not how much of the work it did.
- **Hardware specificity.** The slot counts and decode rates are specific to GB10-class devices with about 273 GB/s of memory bandwidth. A larger local cluster would hide some of these gaps the way the cloud does.
- **A corrected attribution.** Our first diagnosis of the Cognee persist hang cited the wrong upstream issue (§6.3). We report the correction because the episode is itself evidence: without liveness signals, diagnosis falls back on inference.

11 Conclusion

Taking AI on-premises is mostly not about the model. The model was the part that worked: an 80B mixture-of-experts model decoding structured JSON at 138 tok/s across 8 speculative streams.

What failed, in three independent tools, was everything they had assumed about who manages capacity, liveness, retries, request size, model behavior, scope and configuration. The cloud provides

those through a control plane its clients never see. An API-compatible gateway brings back the protocol, which is necessary for anything to connect. It cannot bring back the control plane, and by imitating the cloud it hides the difference until the workload is real.

So the cost of going on-prem is not the hardware or the model. It is the operational work of tuning three layers, the model, your code and your tools, and paying it again with every model swap and every upstream release. A local control plane that states capacity, per-model behavior and configuration authority explicitly keeps that cost bounded, but only for tools written to consult it.

AI coding agents change the price of that work. One operator, working with agents, adapted three cloud-built tools and built the control plane they consult. The cost of going on-prem is still real and still recurring, but it is no longer a reason only large teams can afford to pay.

References

[1] Cognee: open-source AI memory library. `https://github.com/topoteretes/cognee`. Versions: 1.2.2; `main` at `e61bbb79c` and `e8dd3d93d`.

[2] Cognee issue #3870: chat-layer robustness on local backends (fan-out, retry stacking, durability). `https://github.com/topoteretes/cognee/issues/3870`.

[3] Cognee issue #3708: orphaned Kuzu lock after a worker crash; open fixes PR #4033 and PR #4249. `https://github.com/topoteretes/cognee/issues/3708`.

[4] MemPalace fork, branch `kostadis-dev` at `d23d6a7` (upstream v3.10.0); issues and PRs #6, #20, #33, #41, #43, #44, #45, #48, #51, #57 and #59. `https://github.com/kostadis/mempalace`.

[5] dgx-fun working notes: `cognee-local-control-plane-2026-07-04.md`, `cognee-local-control-plane-RETEST-2026-08-01.md`, `cognee-dgx-setup-guide.md`, `mempalace-local-embedder.md`, `dgxlib/ARCHITECTURE.md`.

[6] zvec-grep fork at `73f6991` (upstream `zvec-ai/zvec-grep` v0.2.1). `https://github.com/kostadis/zvec-grep`.